

Unit 8 and 9:

Unit 8. Securing in ASP.NET Core Application

Authentication: ASP.NET Core Identity, Adding authentication to apps and identity service configurations, Authorization: Roles, Claims and Policies, Securing Controllers and Action Methods, Common Vulnerabilities: Cross-site Scripting attacks, SQL Injection attacks, Cross-site Request Forgery (CSRF), Open Redirect Attacks

Unit 9. Hosting and Deploying ASP.NET Core Application

App Servers and Hosting models: IIS, Nginx, Apache, ASP.NET Core Module, Kestrel, Docker and Containerization, Publish to Azure cloud

By: Yuba Raj Devkota
Net Centric Computing
CSIT 6th Semester
NCCS College

Securing ASP.NET Core Applications

Security in ASP.NET Core refers to protecting an application, its users, and its data from unauthorized access and common attacks. It mainly involves:

- *Authentication – verifying who the user is.*
- *Authorization – deciding what an authenticated user is allowed to do.*
- *Data protection – protecting sensitive information such as passwords and tokens.*
- *Input validation – preventing malicious input from reaching the application.*
- *Protection against attacks – such as XSS, SQL Injection, CSRF, and Open Redirect attacks.*

Authentication vs Authorization

Authentication

Verifies the user's identity

"Who are you?"

Usually happens first

Example: Login

Authorization

Determines what the user can access

"What are you allowed to do?"

Happens after authentication

Example: Admin can delete users

ASP.NET Core Identity

ASP.NET Core Identity is a built-in framework for managing users and authentication in ASP.NET Core applications. It provides features such as:

- *User registration*
- *Login and logout*
- *Password management*
- *Password hashing*
- *User roles*
- *Claims*
- *Account confirmation*
- *Two-factor authentication*
- *User information management*

Instead of manually creating all these features, developers can use ASP.NET Core Identity.

Basic Working

```
graph TD; User --> RegisterLogin[Register / Login]; RegisterLogin --> ASPNETCoreIdentity[ASP.NET Core Identity]; ASPNETCoreIdentity --> CheckUsername[Check username & password]; CheckUsername --> UserAuthenticated[User authenticated]; UserAuthenticated --> AccessResources[Access protected resources];
```

User
↓
Register / Login
↓
ASP.NET Core Identity
↓
Check username & password
↓
User authenticated
↓
Access protected resources

Adding Authentication to an ASP.NET Core Application

```
using Microsoft.AspNetCore.Identity;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapDefaultControllerRoute();

app.Run();
```

Authentication can be configured using the Identity service and the authentication middleware. A configuration in `program.cs` file look like this.

This adds ASP.NET Core Identity to the application.

This enables the application to identify or authenticate users.

This enables the application to check whether authenticated users have permission to access a resource.

Authentication should come before authorization because the application needs to know who the user is before deciding what they can access.

Consider a website where only logged-in users can access the Dashboard.

Controller

</> C#

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View();
    }

    [Authorize]
    public IActionResult Dashboard()
    {
        return View();
    }
}
```

The normal `Index()` action can be accessed by anyone.

But:

</> C#

```
[Authorize]
public IActionResult Dashboard()
```

means that the **Dashboard** requires authentication.

If an unauthenticated user tries to access it, ASP.NET Core redirects the user to the login page (depending on the application's authentication configuration).

Authorization in ASP.NET Core

Authorization is the process of determining what an authenticated user is allowed to do in an ASP.NET Core application.

- *Authentication → Who are you?*
- *Authorization → What are you allowed to access?*

For example, after a user logs in, an application may allow:

- *A normal User to view their dashboard.*
- *An Admin to manage users.*
- *A Teacher to manage student records.*

ASP.NET Core provides several ways to implement authorization, mainly **Roles, Claims, and Policies**.

Authorization Using Roles

A **role** represents a category of users, such as `Admin`, `Teacher`, or `Student`.

The `[Authorize]` attribute can be used to restrict access to authenticated users, while `[Authorize(Roles = "...")]` can restrict access to particular roles.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

public class DashboardController : Controller
{
    [Authorize]
    public IActionResult Index()
    {
        return View();
    }
}
```

```
[Authorize(Roles = "Admin")]
public IActionResult AdminPanel()
{
    return View();
}

[Authorize(Roles = "Teacher")]
public IActionResult TeacherPanel()
{
    return View();
}
}
```

Explanation

- `[Authorize]` → Any **authenticated user** can access `Index()` .
- `[Authorize(Roles = "Admin")]` → Only users with the **Admin** role can access `AdminPanel()` .
- `[Authorize(Roles = "Teacher")]` → Only users with the **Teacher** role can access `TeacherPanel()` .

For multiple roles:

`</>` **C#**

```
[Authorize(Roles = "Admin,Teacher")]  
public IActionResult ManageStudents()  
{  
    return View();  
}
```

This allows users who belong to **Admin OR Teacher** role.

Authorization Using Claims

A **claim** is a piece of information about a user.

Examples:

```
Name = Ram  
Email = ram@gmail.com  
Department = IT  
EmployeeType = Teacher
```

Claims are useful when authorization depends on a specific user attribute, rather than simply a role.

Example

Suppose only users whose department is "IT" can access an IT Department

```
[Authorize(Policy = "ITDepartment")]  
public IActionResult ITDashboard()  
{  
    return View();  
}
```

The user's claim might be:

```
Department = IT
```

So, claims provide information that can be used to make authorization decisions.

Authorization Using Policies

A **policy** is a named authorization rule defined by the application.

For example, we can create a policy that requires the user to have the claim:

```
Department = IT
```

Configure the Policy in Program.cs

```
</> C#  
  
builder.Services.AddAuthorization(options =>  
{  
    options.AddPolicy("ITDepartment", policy =>  
        policy.RequireClaim("Department", "IT"));  
});
```

Then use the policy in a controller:

```
</> C#  
  
[Authorize(Policy = "ITDepartment")]  
public IActionResult ITDashboard()  
{  
    return View();  
}
```

Only users having the Department claim with value IT can access the action.

Example: Policy Authorization for Voting (Age 18+)

Suppose only users who are **18 years or older** can access the voting page.

Program.cs

```
</> C#  
  
builder.Services.AddAuthorization(options =>  
{  
    options.AddPolicy("CanVote", policy =>  
        policy.RequireClaim("Age", "18", "19", "20"));  
});
```

In real applications, age-based policies can be implemented more flexibly using custom authorization requirements.

VotingController.cs

```
</> C#  
  
using Microsoft.AspNetCore.Authorization;  
using Microsoft.AspNetCore.Mvc;  
  
public class VotingController : Controller  
{  
    [Authorize(Policy = "CanVote")]  
    public IActionResult Vote()  
    {  
        return View();  
    }  
}
```

Securing Controllers and Action Methods

Securing the Entire Controller

```
</> C#  
  
[Authorize]  
public class StudentController : Controller  
{  
    public IActionResult Index()  
    {  
        return View();  
    }  
  
    public IActionResult Profile()  
    {  
        return View();  
    }  
}
```

Allowing One Action Without Authentication

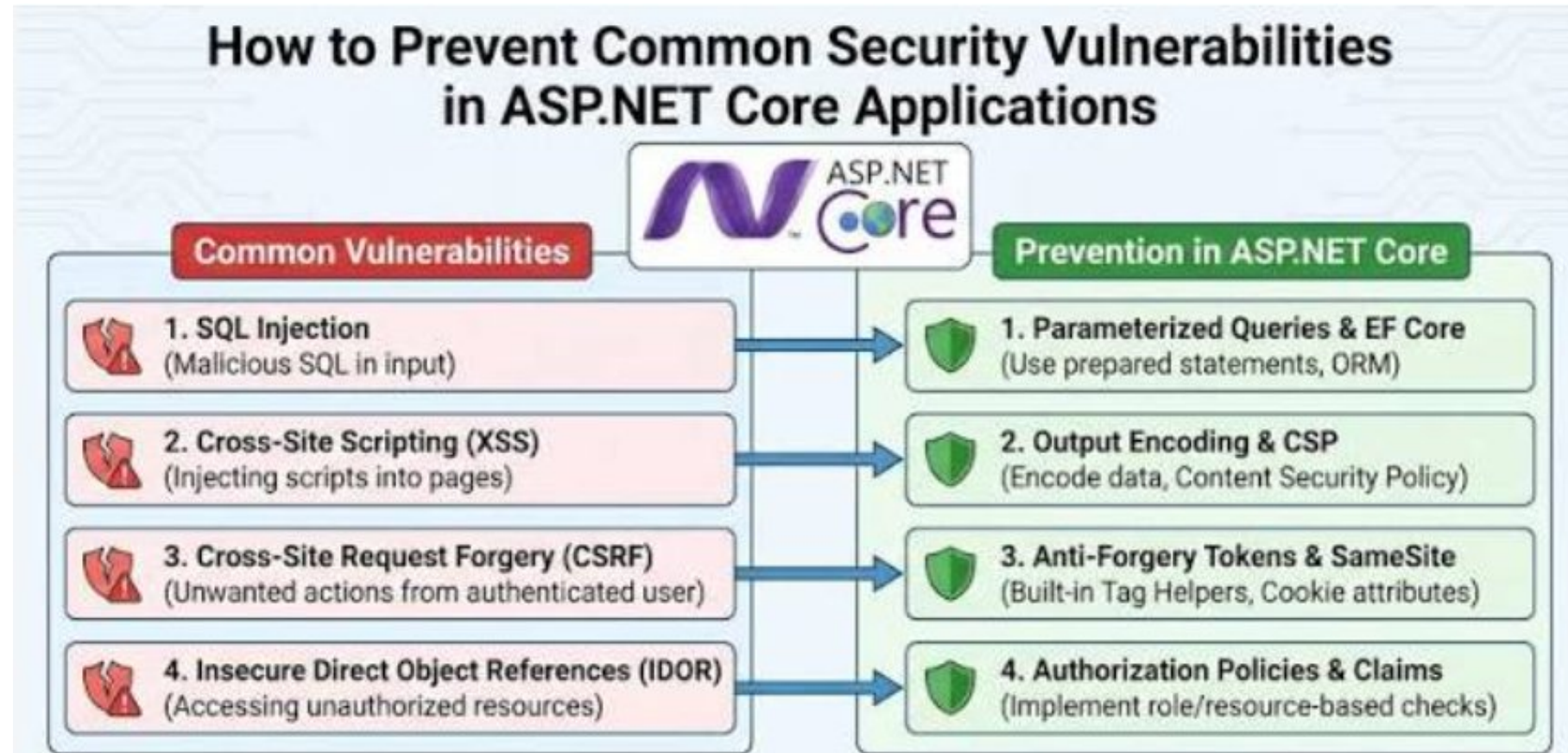
```
[Authorize]  
public class StudentController : Controller  
{  
    public IActionResult Dashboard()  
    {  
        return View();  
    }  
  
    [AllowAnonymous]  
    public IActionResult About()  
    {  
        return View();  
    }  
}
```

- `Dashboard()` → Requires authentication.
- `About()` → Can be accessed by anyone.

Common Vulnerabilities in ASP.NET Core Applications

A vulnerability is a weakness in an application that attackers can exploit to access data, modify information, or perform unauthorized actions. The common web application vulnerabilities in this unit are:

- *Cross-Site Scripting (XSS)*
- *SQL Injection (SI)*
- *Cross-Site Request Forgery (CSRF)*
- *Open Redirect (OR)*



1. Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) occurs when an attacker inserts malicious JavaScript into a web page, causing the script to execute in another user's browser.

Example: Student Comment System

Suppose an application allows students to post comments.

Vulnerable code

Controller:

```
</> C#  
  
[HttpPost]  
public IActionResult AddComment(string comment)  
{  
    ViewBag.Comment = comment;  
    return View();  
}
```

View:

```
HTML  
  
<h3>Student Comment</h3>  
  
<div>  
    @Html.Raw(ViewBag.Comment)  
</div>
```

A normal student enters:

This is a very useful course.

The page displays:

Student Comment
This is a very useful course.

However, an attacker enters:

HTML

```
<script>
    alert("Your account has been attacked!");
</script>
```

Because `Html.Raw()` tells ASP.NET Core to render the input as HTML, the browser executes the JavaScript.

Safer approach

Use normal Razor output:

HTML

```
<div>
    @ViewBag.Comment
</div>
```

ASP.NET Core Razor automatically HTML-encodes the output, so the malicious script is treated as text instead of being executed.

XSS → Malicious script is inserted into a webpage and executed in the user's browser.

2. SQL Injection

SQL Injection occurs when an attacker inserts malicious SQL into input fields and the application directly combines that input with an SQL query.

Example: Student Login System

Suppose a student login page has:

Username:

Password:

[Login]

For a normal login:

Username: ram

Password: abc123

The developer writes:

```
string query = "SELECT * FROM Students " +  
               "WHERE Username = '" + username +  
               "' AND Password = '" + password + "'";
```

The query becomes:

</> SQL

```
SELECT * FROM Students  
WHERE Username = 'ram' AND Password = 'abc123'
```

This works normally.

Attacker's input

Instead of entering a normal username, the attacker enters:

```
' OR '1'='1
```

The generated query can become logically different from what the developer intended:

`</>` SQL

```
SELECT * FROM Students
WHERE Username = '' OR '1'='1'
AND Password = ''
```

The condition `'1'='1'` is always true, potentially allowing unauthorized access depending on the application's query and authentication logic.

Safer approach

Do not concatenate user input into SQL. Use **Entity Framework Core** or parameterized queries.

For example:

</> C#

```
var student = db.Students
    .FirstOrDefault(s => s.Username == username &&
        s.Password == password);
```

Here, the user's input is treated as **data**, not as part of the SQL command.

Remember:

SQL Injection → Malicious input changes the database query.

Parameterized Query to Prevent SQL Injection

Instead of directly concatenating user input into an SQL query:

`</> C#`

```
string query = "SELECT * FROM Students WHERE Username = '"  
              + username + "'";
```

Here @username is a parameter. The user's input is treated as data, not as part of the SQL command.

use a **parameterized query**:

`</> C#`

```
string query = "SELECT * FROM Students WHERE Username = @username";  
  
using SqlCommand cmd = new SqlCommand(query, connection);  
cmd.Parameters.AddWithValue("@username", username);  
  
SqlDataReader reader = cmd.ExecuteReader();
```

For example, even if the user enters:

```
' OR '1'='1
```

it is treated as a username value rather than executable SQL.

3. Cross-Site Request Forgery (CSRF)

CSRF occurs when an attacker tricks an already authenticated user into sending an unwanted request to a website.

Example: Changing Student Email

Suppose a student is logged into a college portal.

The portal provides a form:

Change Email

New Email: [_____]

[Change Email]

The controller contains:

```
</> C#
```

```
[HttpPost]
```

```
public IActionResult ChangeEmail(string email)
```

```
{
```

```
    // Change the logged-in student's email
```

```
    return RedirectToAction("Profile");
```

```
}
```

The normal form is:

HTML

```
<form method="post" action="/Student/ChangeEmail">  
  <input type="email" name="email">  
  <button type="submit">Change Email</button>  
</form>
```

The student is already logged into:

college.edu

An attacker could create another website containing a hidden form that submits a request to:

college.edu/Student/ChangeEmail

with the attacker's email address.

If the application does not have CSRF protection, the browser may send the request using the student's existing login session.

Prevention in ASP.NET Core

Add an **Anti-Forgery Token** to the form:

HTML

```
<form method="post" action="/Student/ChangeEmail">
    @Html.AntiForgeryToken()

    <input type="email" name="email">
    <button type="submit">Change Email</button>
</form>
```

Then protect the action:

C#

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult ChangeEmail(string email)
{
    // Change email
    return RedirectToAction("Profile");
}
```

ASP.NET Core checks the anti-forgery token before accepting the request.

Remember:

CSRF → Attacker tricks a logged-in user into sending an unwanted request.

4. Open Redirect Attack

An Open Redirect occurs when an application redirects a user to a URL supplied by the user without properly validating it.

Example: Login Page

Suppose a website sends the user to the login page when authentication is required.

The URL might look like:

```
https://college.edu/login?returnUrl=/Student/Dashboard
```

After successful login, the application wants to send the student back to the requested page.

A developer writes:

```
</> C#  
  
public IActionResult Login(string returnUrl)  
{  
    // Login successful  
  
    return Redirect(returnUrl);  
}
```

For a normal request:

```
returnUrl=/Student/Dashboard
```

The user is redirected to:

```
https://college.edu/Student/Dashboard
```

Attacker's URL

An attacker can create a link such as:

```
https://college.edu/login?returnUrl=https://malicious-site.com
```

The user may trust the link because it starts with:

```
https://college.edu
```

After login, however, the application redirects the user to:

```
https://malicious-site.com
```

The attacker could use this technique in a phishing attack to make a malicious link appear to originate from a trusted website.

Prevention in ASP.NET Core

Check whether the URL is a local URL:

</> C#

```
public IActionResult Login(string returnUrl)
{
    // Login successful

    if (Url.IsLocalUrl(returnUrl))
    {
        return Redirect(returnUrl);
    }

    return RedirectToAction("Index", "Home");
}
```

Remember:

Open Redirect → Attacker supplies a URL and the application redirects the user to it.

`Url.IsLocalUrl()` helps ensure that the destination is within the same application.

Summary

Vulnerability	Example	Prevention
XSS	Attacker puts JavaScript in a student comment	Output encoding; avoid unsafe <code>Html.Raw()</code>
SQL Injection	Attacker manipulates a login/database query	EF Core / parameterized queries
CSRF	Attacker tricks logged-in student into changing email	Anti-Forgery Token
Open Redirect	Attacker makes login redirect to a malicious website	Validate URL using <code>Url.IsLocalUrl()</code>

- *XSS attacks the browser*
- *SQL Injection attacks the database*
- *CSRF abuses the user's authenticated session, and*
- *Open Redirect abuses the application's redirection mechanism.*

Example to show the possibility of SQL injection attack

Suppose we have a login form with fields for **username** and **password**.

The server-side code (insecure) is:

```
string query = "SELECT * FROM Users WHERE Username = '"  
                + txtUsername.Text + "' AND Password = '"  
                + txtPassword.Text + "'";
```

If a user enters:

- Username: admin
- Password: ' OR '1'='1

```
SELECT * FROM Users WHERE Username = 'admin' AND Password = '' OR '1'='1';
```

The query becomes:

Explanation

- ' OR '1'='1 always evaluates to **true**.
- The database returns a valid row, so the attacker can log in **without knowing the real password**.
- This allows unauthorized access and may expose sensitive data.

In short: SQL Injection happens when user input is directly concatenated into SQL queries without validation. Attackers can manipulate queries to bypass authentication or steal data.

How do you prevent SQL Injection attack?

✅ Preventing SQL Injection Attacks

The best way to prevent SQL Injection is to **never directly concatenate user input into SQL queries**. Instead, use **parameterized queries (prepared statements)** which safely bind user inputs as parameters.

Other measures include:

1. Parameterized Queries / Stored Procedures
2. Input Validation & Sanitization
3. Least Privilege Access (DB accounts should have limited rights)
4. Use of ORM frameworks (like Entity Framework, Hibernate)

Unsafe Code (Vulnerable to SQL Injection)

csharp

```
string query = "SELECT * FROM Users WHERE Username = '"  
                + txtUsername.Text + "' AND Password = '"  
                + txtPassword.Text + "'";  
  
SqlCommand cmd = new SqlCommand(query, conn);
```

If attacker enters `' OR '1'='1`, it bypasses login. 🚨

Safe Code using Parameterized Query

```
string query = "SELECT * FROM Users WHERE Username = @username AND Password = @password";  
SqlCommand cmd = new SqlCommand(query, conn);  
  
cmd.Parameters.AddWithValue("@username", txtUsername.Text);  
cmd.Parameters.AddWithValue("@password", txtPassword.Text);
```

Here, even if the user enters ' OR '1'='1', it is treated as **data**, not as part of the SQL command. 

In short:

- **Problem:** Direct string concatenation → SQL Injection possible.
- **Solution:** Use **parameterized queries** or stored procedures.